

Personal Email RAG

ARCHITECTURE NOTE | Local inference, traceable evidence, scoped access

1. System boundary and data flow

The application is a trusted local Python CLI. It accepts RFC-style EML/MBOX exports or reads a real Gmail account using explicit, read-only OAuth consent. Google is contacted only as a mailbox data source. Email extraction, embeddings, vector search and answer generation execute on the local machine; there are no OpenAI or Claude inference calls.

Stage	Component and contract
Ingest	Python email parser; Gmail raw MIME; PDF/DOCX/text extractors
Vectorize	Ollama Nomic embeddings; 768 floats per passage
Persist	PostgreSQL 16 + pgvector; emails and tenant-scoped chunks
Retrieve	Token identity; SQL tenant filters + RLS; cosine/full-text ranking
Answer	Local Mistral; bounded evidence; [S#] labels and source links

2. Parsing, attachment coverage and incremental writes

MIME alternatives prefer plain text; HTML drops scripts/styles. Sender, recipient, subject and UTC timestamp are preserved. Supported attachments become separate searchable passages with attachment-name metadata and the original email link. Unsupported, encrypted or non-text documents produce visible warnings; extraction completeness is never assumed. Body and attachment text use 1,200-character chunks with 180-character overlap and bounded headers.

Message identity is a hash of Message-ID (or raw content when absent); Gmail uses account plus API message ID. A separate content hash skips unchanged input. Updated messages replace all chunks transactionally after embedding succeeds. Content-addressed EML files preserve checkable local sources; Gmail uses account-specific thread links.

3. Embedding and generation model choices

Nomic embed text v1.5 provides compact local retrieval embeddings. The code uses `search_document/search_query` prefixes, requires 768 finite nonzero values and disables silent input truncation. The index stores the model tag plus digest so changed vector spaces cannot be mixed. The supplied variant uses F16 weights [2].

Mistral 7B Instruct v0.3, Q4_K_M balances local deployment cost with instruction-following ability. Its Apache-2.0 model variant is approximately 4.4 GB [1]; runtime memory also includes context/KV cache and is higher. Four-bit quantization reduces weight memory at a possible quality cost. Generation uses temperature 0, a 16,384-token context setting and a 700-token output cap. Quality and hardware latency remain to be measured.

Retrieval, isolation & evaluation

4. Vector database and retrieval configuration

pgvector is the preferred database in the brief and allows vectors, timestamps, metadata, transactions and row policies in one store. Chunks use vector(768), composite (user_id, email_id, chunk_no) keys and cosine distance. The sample uses exact scans rather than ANN to avoid recall loss after tenant filtering. A B-tree supports user/date lookup. HNSW is a future scaling option after profiling, not a mandatory component for a ten-message corpus.

Default relevance combines cosine similarity with a small full-text bonus. Explicit sender/subject filters apply before ranking. Latest mode requires a sender filter and sorts all matching dated rows first. The all-matching summary mode reads every chunk in a declared subject scope, then generates chronological batch summaries with globally unique citations. A 500-chunk limit fails visibly rather than hiding partial coverage.

5. Authentication and multi-user boundary

Each local user has a random capability token; only its SHA-256 hash is stored in the users table. A validated token maps to a UUID. The application never accepts an LLM-supplied tenant selector. Queries set that UUID transaction-locally; forced row-level policies independently restrict reads and writes. The application database role has no superuser/BYPASSRLS powers. Alice and Bob have distinct synthetic datasets and token files.

Boundary: RLS guards application queries, not a hostile operator with database credentials or access to another token file. A shared local OS administrator remains trusted. Public multi-user deployment would need a trusted service, stronger session identity, separate credential custody, encryption, rate controls and audits. Gmail OAuth authorizes mailbox access separately from application capability tokens.

6. Grounding, failure behavior and security

Ollama calls allow only loopback origins, disable proxies/redirects and reject cloud tags; the documented service configuration disables cloud features. Retrieved evidence is labeled untrusted, and the model receives no tools or mail-writing functions. Source-label checks warn about missing or unknown citations, but do not prove entailment or defeat every prompt injection. Empty retrieval skips generation. Per-stage timings and extraction warnings are returned with results.

7. Verification status and trade-offs

Offline parsing/orchestration tests and parser timing measurements ran in the authoring environment. Actual database RLS, local model quality/latency and real Gmail ingestion still require the target machine. The repository includes eight opt-in database tests, a five-case live benchmark, a demo script and an evaluation report that separates measured evidence from pending work. Google Drive ingestion, OCR and production hosting are outside this implementation.

Primary references

- [1] ollama.com/library/mistral:7b-instruct-v0.3-q4_K_M
- [2] ollama.com/library/nomic-embed-text:v1.5
- [3] github.com/pgvector/pgvector; postgresql.org/docs/16/ddl-rowsecurity.html
- [4] docs.ollama.com/api/embed; docs.ollama.com/api/generate
- [5] developers.google.com/workspace/gmail/api/quickstart/python